Embeddings and Semantic Search

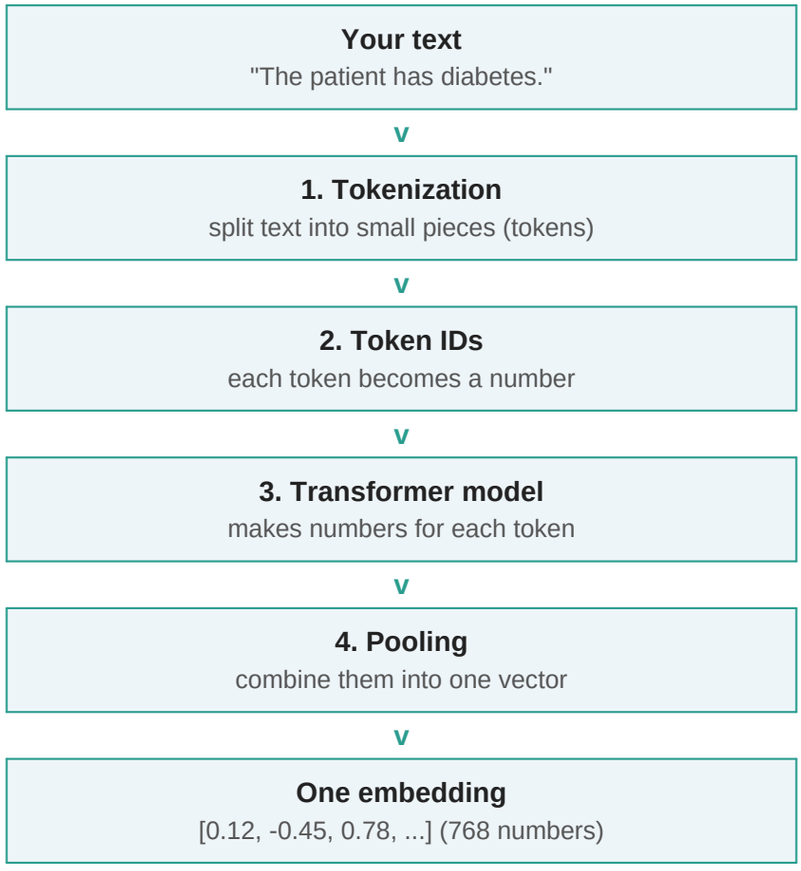
Beginner Notes: how `model.encode()` works

> **In one line:** An embedding turns text into a list of numbers, so a computer can compare meanings using maths.

1. The big picture

When you write `model.encode("The patient has diabetes.")`, many steps run for you automatically. You only write one line of code:

```
embedding = model.encode(text)
```



2. What is an embedding?

An embedding is a **list of numbers that represents something in a mathematical space**. There is nothing magical about the word. A 2-number embedding is 2-dimensional. Your model makes 768 numbers, so it is 768-dimensional. The maths works the same way, but we cannot draw 768 dimensions.

Tiny 2D example (made up for teaching):

Word	Embedding
cat	[0.9, 0.8]
dog	[0.8, 0.9]
car	[0.1, 0.2]
banana	[0.7, 0.1]

On a graph, cat and dog sit close together, and car sits far away. Close points mean related meaning. This is an **embedding space**.

3. Where does meaning come from?

In our toy example we *decided* that cat and dog are close. A real model **learns** this from huge amounts of text. It sees sentences like "The cat is sleeping" and "The dog is sleeping" again and again, and adjusts its parameters during training. After training, related words and sentences end up in nearby places:

- **Close:** cat, dog, kitten, puppy
- **Far:** cat, engine, database

> **Key idea:** The real magic is not "768 numbers". It is how the model *learned* numbers that are useful for meaning.

4. A tiny semantic program (toy version)

This is **not** how a Transformer works. We make the vectors by hand, just to understand the idea. Each word gets 3 scores: animal, vehicle, food.

Word	animal	vehicle	food
cat / dog / tiger	1	0	0
car / bus	0	1	0
pizza / banana	0	0	1

Cosine similarity tells us how similar two vectors are. Score 1.0 means same direction (very similar). Score 0.0 means unrelated.

```
import math

def cosine_similarity(a, b):
    dot = sum(x * y for x, y in zip(a, b))
    length_a = math.sqrt(sum(x * x for x in a))
    length_b = math.sqrt(sum(x * x for x in b))
    return dot / (length_a * length_b)

words = {
    "cat": [1, 0, 0], "dog": [1, 0, 0], "tiger": [1, 0, 0],
    "car": [0, 1, 0], "bus": [0, 1, 0],
    "pizza": [0, 0, 1], "banana": [0, 0, 1],
}

def search(query):
    q = words[query]
    results = [(w, cosine_similarity(q, v))
               for w, v in words.items() if w != query]
    return sorted(results, key=lambda x: x[1], reverse=True)

print(search("cat"))
# dog 1.0, tiger 1.0, car 0.0, bus 0.0, pizza 0.0, banana 0.0
```

You just built a tiny semantic search engine. Real meaning is richer than 3 labels. For example, with scores for *animal, living, domestic*, cat = [1,1,1] and tiger = [1,1,0] are related but not identical (similarity about 0.82).

> **Remember:** In real embeddings, dimensions have **no human names**. You cannot say "dimension 17 means medical". The meaning is spread across many dimensions, like a complicated coordinate system.

5. Tokenization: text to pieces

A model cannot read English directly. First, a **tokenizer** splits the text into tokens. Real tokenizers do not always split on spaces. They use **subword tokenization**:

```
"The patient has diabetes."
simple split : ["The", "patient", "has", "diabetes", "."]
subword split: ["The", "patient", "has", "di", "abetes", "."]
               (exact split depends on the tokenizer)
```

Why not just use `text.split()`? Because of words like *unbelievable, COVID-19, hypertension*. The tokenizer may not know them exactly, so it breaks them into known pieces, for example *unhappiness* = *un* + *happi* + *ness*.

6. Token IDs

The tokenizer has a vocabulary (a big list). Each token is replaced by its position number in that list, called a **token ID**:

```
"The" -> 101     "patient" -> 5821
"has"  -> 1034    "diabetes" -> 7634
"The patient has diabetes."  ->  [101, 5821, 1034, 7634]
```

> **Careful:** A token ID like 5821 is only an index in the vocabulary. It does **not** mean anything by itself.

7. The Transformer: context matters

The Transformer receives the token IDs and creates a vector for each token. Its special power is that each vector depends on the **context** (the words around it). Look at the word "bank":

- "I deposited money in the **bank**." (a place for money)
- "I sat beside the river **bank**." (the side of a river)

Same spelling, different meaning. The Transformer gives them different vectors because the surrounding words are different.

8. Pooling: many vectors to one

Now we have one vector per token, but we want **one vector for the whole sentence**. Pooling does this. Your model uses **mean pooling**, which simply averages all token vectors. Teaching example with 3 dimensions:

Token	Dim 1	Dim 2	Dim 3
The	0.2	0.1	0.3
patient	0.8	0.7	0.9
has	0.1	0.2	0.1
diabetes	0.9	0.8	0.7
Average (sentence)	**0.50**	**0.45**	**0.50**

Example for Dim 1: (0.2 + 0.8 + 0.1 + 0.9) / 4 = 0.5. Your real model does the same thing, but with 768 dimensions.

9. Putting it together with sentences

```
1: "The patient has diabetes."
   -> [ 0.12, -0.43,  0.81, ...,  0.21]
2: "The patient suffers from diabetes."
   -> [ 0.15, -0.39,  0.79, ...,  0.18]
3: "The car is parked outside."
   -> [-0.71,  0.21, -0.08, ...,  0.62]
```

- Sentence 1 and 2 have similar meaning, so their vectors are **close**.
- Sentence 1 and 3 are unrelated, so their vectors are **far apart**.

10. Semantic search and RAG

This is the main use of embeddings. Imagine 1 million medical documents:

- Turn every document into a 768-number vector (done once, in advance).
- A user searches "diabetes treatment". Turn the query into a vector too.
- Compare the query vector with every document vector using cosine similarity.
- The highest scores are the most relevant documents.

This is called **vector search**, and it is the base of **RAG** (Retrieval-Augmented Generation).

11. Toy program vs real AI

	Our toy program	Real embedding model
Who makes the vectors?	A human, by hand	The model, by learning
How?	Fixed rules	Big dataset, neural network, training, millions or billions of parameters
Result	Only a few simple labels	Rich meaning in 768 dimensions

This connects to normal machine learning ideas:

```
Data -> Training -> Parameters -> Loss
  -> Optimization -> Learned representation -> Embedding
```

The embedding is the neural network's **learned representation** of the text.

12. Who does what? (the tools)

Tool	Question it answers
Hugging Face Hub	Where is the model stored and downloaded from?
Transformer	How does the neural network process text?
Sentence Transformers (library)	How do I easily get sentence embeddings?
Cosine similarity / vector search	How do I compare meanings?
Semantic search / RAG	How do I use all this in a real application?

13. Quick summary

- An **embedding** = a list of numbers that stands for the meaning of text.
- **Pipeline:** tokenize, token IDs, Transformer, pooling, one vector.
- All of this happens automatically inside `model.encode()`.

- **Similar meaning = close vectors**, measured with cosine similarity.
- The model **learns** the numbers from data. Dimensions have no human names.
- Embeddings power semantic search and RAG.

> **Next step:** Take one real sentence, such as "The patient has diabetes", and inspect the real model at each stage: actual tokens, token IDs, tensor shapes, the 768-number output, and cosine similarity between two real sentences.